

GCC1734 – Inteligência Artificial
Nota de Aula 05
Busca Adversarial: Monte Carlo Tree Search

Prof. Eduardo Bezerra

Sumário

1	Introdução	3
1.1	Em direção ao aprendizado	4
1.2	Limitações do Minimax e Motivação para o MCTS	5
1.3	Horizonte fixo	5
1.4	Dependência de funções de avaliação específicas ao domínio	5
1.5	O que o MCTS faz de diferente	5
2	Estrutura de Dados da Árvore MCTS	6
2.1	Exemplo de árvore com contadores	7
3	O Algoritmo MCTS: Quatro Passos	9
3.1	Passos do algoritmo	10
3.1.1	Passo 1: Select (Seleção)	10
3.1.2	Passo 2: Expand (Expansão)	10
3.1.3	Passo 3: Simulate (Simulação)	11
3.1.4	Passo 4: Back-Propagate (Retropropagação)	12
3.2	Como o MCTS começa?	13
3.3	Exemplo resolvido: uma iteração completa	15
3.4	Exemplo resolvido: árvore reduzida inspirada no Jogo da Velha	17
3.5	Critério de parada e resultado final	20
4	Política de Seleção e a Heurística UCB1	21
4.1	O dilema exploração-exploração	21
4.2	A heurística UCB1	22
4.3	Exemplo resolvido: calculando UCB1	24
5	Política de Simulação	26
5.1	Política de simulação aleatória	26
5.2	Políticas de simulação baseadas em heurísticas	27
5.3	Políticas aprendidas por self-play	27
6	Resumo e Comparação com o Minimax	28

7	Atividades Intra-Classe	30
7.1	Aquecimento	30
7.2	Exercício: simulação manual do MCTS	30
7.3	Exercício: UCB1 e o dilema exploração-exploração	31
7.4	Projeto de política de simulação	31

Objetivos

Ao final da leitura, você deve ser capaz de:

Motivação

- Explicar as limitações do Minimax e de suas variantes que motivam o uso do MCTS.
- Identificar as classes de jogos para as quais o MCTS é especialmente adequado.

Estrutura de dados ★

- Descrever a estrutura de dados mantida pelo MCTS e o significado dos contadores $N(s)$ e $W(s)$.
- Calcular a estimativa de valor $Q(s) = W(s)/N(s)$ e interpretar seu significado probabilístico.

Algoritmo ★

- Descrever os quatro passos do ciclo MCTS e a função de cada um: Select, Expand, Simulate e Back-Propagate.
- Executar manualmente uma iteração completa do ciclo MCTS sobre uma árvore concreta, atualizando os contadores corretamente.
- Explicar como o resultado final (ação escolhida) é extraído ao término das iterações.

Políticas de seleção e de simulação ★

- Enunciar o dilema exploração-exploração e explicar por que ele é central para a eficiência do MCTS.
- Definir a heurística UCB1, calcular seu valor para nós concretos e identificar qual nó ela seleciona.
- Descrever o papel da política de simulação e contrastar a abordagem de heurísticas manuais com o aprendizado por self-play.

Esta leitura complementa as notas sobre Minimax. Ela começa pelas dificuldades do Minimax em jogos de grande escala e pela alternativa proposta pelo MCTS (Seções 1–2), passa pela estrutura da árvore MCTS (Seção 3) e pelo seu ciclo de quatro passos (Seção 4), e termina com as políticas que governam o algoritmo: a política de seleção com UCB1 (Seção 5) e a política de simulação, incluindo self-play (Seção 6). Os objetivos marcados com ★ são os mais centrais e devem receber prioridade se o tempo for limitado.

1 Introdução

Nesta nota, consideramos a formulação clássica de busca adversarial: jogos finitos, determinísticos, de dois jogadores, soma zero, informação perfeita e turnos alternados. Extensões para jogos estocásticos, informação imperfeita, múltiplos jogadores ou objetivos não estritamente opostos exigem adaptações no valor armazenado, na política de seleção e na retropropagação.

1.1 Em direção ao aprendizado

Até agora, estudamos algoritmos que calculam a melhor decisão: o agente conhece as regras do jogo, explora a árvore de possibilidades e escolhe a ação com melhor valor, como no Minimax. Nessa abordagem, o agente não precisa aprender; ele apenas aplica regras e executa a busca. O MCTS segue uma lógica diferente: em vez de calcular diretamente o valor dos estados, joga várias partidas a partir do estado atual, observa seus resultados e usa essa evidência para guiar decisões futuras. Há, portanto, uma mudança de perspectiva: em vez de decidir apenas por raciocínio sobre a árvore, o agente também decide experimentando.

A Figura 1 resume a transição conceitual que começa nesta aula. Nos métodos de busca adversarial clássicos, como o H-Minimax, a árvore é explorada até certa profundidade e os estados não terminais são avaliados por uma função heurística construída manualmente. O MCTS mantém a ideia de busca em árvore, mas substitui essa avaliação heurística explícita por estimativas obtidas a partir de simulações. Essa mudança prepara o terreno para métodos mais modernos, nos quais política e valor deixam de ser projetados manualmente e passam a ser aprendidos a partir de dados.

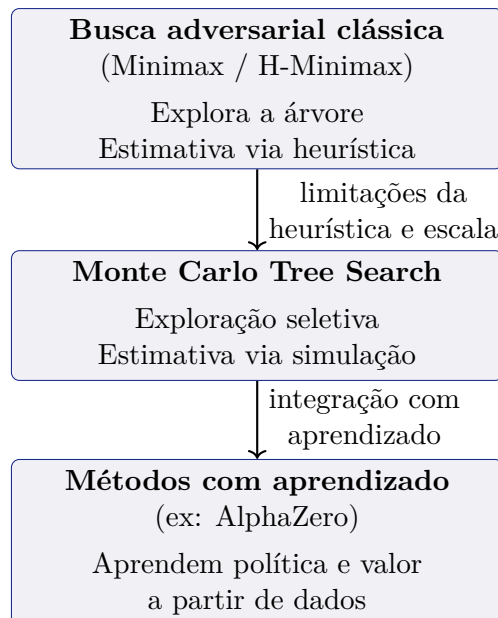


Figura 1: Evolução conceitual dos métodos de decisão em jogos. A diferença central não está em estimar ou não estimar, mas em *como* a estimativa é obtida: heurísticas manuais, simulações ou aprendizado a partir de dados.

O nome “Monte Carlo” remete ao método estatístico de estimar quantidades difíceis de calcular analiticamente por meio de amostragem aleatória. No MCTS, o valor de um estado é estimado pela *média dos resultados* de muitas simulações a partir dele, em vez de ser calculado exatamente pela retropropagação de valores terminais como no Minimax completo. Em geral, quanto mais simulações forem realizadas, mais confiável se torna essa estimativa, assim como uma pesquisa de opinião tende a se tornar mais precisa quando consulta mais pessoas.

1.2 Limitações do Minimax e Motivação para o MCTS

As notas anteriores apresentaram o Minimax e sua otimização via poda alfa-beta. Esse arcabouço é elegante e produz decisões ótimas contra oponentes racionais em jogos pequenos. No entanto, duas limitações fundamentais tornam o Minimax e suas variantes inadequados para jogos de grande escala.

1.3 Horizonte fixo

O Minimax expande a árvore de jogo até uma profundidade d predefinida e aplica uma função de avaliação heurística nos nós folha. Essa profundidade é fixada antes da busca e não se adapta à complexidade do estado. O problema: em jogos com fator de ramificação elevado, aumentar d em apenas um nível multiplica o trabalho por b , pois o número de nós explorados cresce como $O(b^d)$.

No Go, por exemplo, o fator de ramificação é $b > 300$ (há mais de 300 jogadas legais em uma posição típica). Comparativamente, o xadrez tem $b \approx 35$. Com esse fator de ramificação, qualquer profundidade razoável gera uma quantidade de nós que excede em muito a capacidade computacional disponível. O resultado: o agente enxerga apenas um número pequeno de lances à frente, insuficiente para capturar planos estratégicos de médio e longo prazo.

1.4 Dependência de funções de avaliação específicas ao domínio

Como a busca não alcança estados terminais, o Minimax depende de uma função de avaliação heurística $EVAL(s)$ para estimar o valor de estados intermediários. Projetar essa função exige conhecimento especializado do domínio e pode introduzir viés ou fragilidade: se a função não capturar adequadamente a estratégia relevante, o agente tomará decisões sistematicamente ruins.

No xadrez, funções de avaliação razoavelmente boas foram desenvolvidas por décadas de trabalho de especialistas (contagem de material, controle do centro, segurança do rei, etc.). No Go, essa tarefa se mostrou muito mais difícil: o valor de uma posição depende de padrões sutis de influência territorial que são difíceis de formalizar em uma soma ponderada de características simples.

1.5 O que o MCTS faz de diferente

O algoritmo *Monte Carlo Tree Search* (MCTS) contorna essas duas limitações de forma elegante. A ideia central é substituir as operações mais custosas do Minimax por alternativas mais escaláveis.

Em relação ao *horizonte fixo*, em vez de expandir a árvore até uma profundidade d com avaliação heurística nas folhas, o MCTS executa *simulações* completas a partir de estados intermediários. Uma simulação consiste em jogar a partida do estado atual até o seu término natural (vitória, derrota ou empate) seguindo uma política simples de escolha de ações. O resultado terminal de uma partida simulada (e não uma estimativa heurística manual) é então usado como sinal de avaliação daquele estado. Isso elimina a necessidade de projetar uma $EVAL(s)$ e, ao mesmo tempo, contorna o problema do horizonte fixo: cada simulação alcança o estado terminal independentemente da profundidade do jogo.

Em relação à *dependência de funções heurísticas*, o MCTS não expande a árvore de forma uniforme até profundidade d . Em vez disso, a árvore cresce de forma *assimétrica*:

regiões que parecem promissoras recebem mais simulações, enquanto regiões que parecem fracas são visitadas com menor frequência. Essa adaptação dinâmica é governada por uma *política de seleção* que equilibra dois objetivos: explorar regiões novas e aproveitar regiões já conhecidas como boas. Um boa política faz isso sem precisar de qualquer conhecimento específico do domínio.

O MCTS constrói uma árvore de jogo de forma incremental, iteração a iteração. Em vez de expandir a árvore inteira até uma profundidade fixa, como faz o Minimax, o MCTS acrescenta *um nó por vez* e concentra o esforço nas regiões que parecem mais promissoras. Com isso, evita materializar todos os b^d nós de uma árvore uniforme; ainda assim, cada iteração paga o custo de percorrer uma seleção, adicionar um nó e executar um *playout* cuja profundidade pode ser grande. O valor de cada estado não é estimado por uma função heurística: é inferido a partir dos resultados de partidas simuladas até o fim, acumulados em valores armazenados nos nós da árvore. Ao final de todas as iterações, o MCTS retorna a ação que leva ao filho da raiz mais visitado, aquele cuja estimativa é estatisticamente mais confiável.

Pergunta 1

Quais são as duas desvantagens do Minimax que o MCTS busca superar? Para cada uma delas, explique brevemente o que o MCTS faz de diferente.

Pergunta 2

Por que o fator de ramificação b tem impacto tão severo sobre a viabilidade do Minimax? Considere dois jogos: um com $b = 35$ (xadrez) e um com $b = 300$ (Go). Se ambos tiverem profundidade $d = 6$, quantos nós cada um expande? O que esse cálculo revela sobre a escalabilidade do Minimax?

Outra consequência desse crescimento incremental é que o MCTS é naturalmente *any-time*: ele pode ser interrompido a qualquer momento e retorna a melhor ação encontrada até ali. Ao contrário do Minimax com profundidade fixa, não há um horizonte predefinido; mais tempo permite refinar as estimativas. Isso o torna especialmente adequado para situações com restrições de tempo variáveis.

2 Estrutura de Dados da Árvore MCTS

O MCTS mantém uma árvore de jogo na memória. Cada nó dessa árvore representa um estado do jogo e armazena duas quantidades:

- $N(s)$: o número inteiro de simulações já realizadas que *passaram pelo estado s* ao longo do caminho raiz-folha. Em particular, $N(s)$ é incrementado tanto quando o nó s foi o ponto de partida de uma simulação, quanto quando foi simplesmente atravessado durante a fase de seleção antes de se chegar a um nó mais profundo.
- $W(s)$: a soma acumulada dos retornos de MAX nas simulações que passaram por s . Em jogos sem empate, esse valor coincide com o número de vitórias de MAX; se empates valem 0,5, $W(s)$ pode ser fracionário.

Independentemente de o nó s ser MAX ou MIN, $W(s)$ registra a soma dos retornos de MAX nas simulações que passaram por s . Essa convenção uniforme simplifica o Back-

Propagate, pois a atualização pode percorrer o caminho da folha até a raiz aplicando a mesma regra, sem verificar qual jogador decide em cada nó. Em contrapartida, a interpretação de $Q(s) = W(s)/N(s)$ exige cuidado: um valor alto em um nó MIN não significa que MIN está em boa posição, mas que MAX tende a vencer quando o jogo passa por esse estado. A Seção 4 mostrará como a política de seleção trata essa assimetria.

A partir desses dois contadores, define-se a estimativa de valor do estado s :

$$Q(s) = \frac{W(s)}{N(s)}$$

$Q(s)$ representa o retorno médio de MAX nas simulações passando por s . Quando os únicos resultados possíveis são vitória (+1) ou derrota (0), $Q(s)$ coincide com a porcentagem de vitórias de MAX a partir de s . De forma mais geral, com empates ou recompensas intermediárias, $Q(s)$ é a *utilidade média* das simulações passando por s .

$Q(s)$ pode ser lido como uma estimativa da probabilidade de vitória de MAX somente no caso binário vitória/derrota e sob uma política fixa de continuação. No MCTS, a política abaixo de um nó muda à medida que a árvore cresce; por isso, mais precisamente, $Q(s)$ é a média empírica dos retornos obtidos pela política de busca e simulação vigente ao longo das iterações. Com poucas simulações, essa quantidade é ruidosa e pode estar distante do valor minimax do estado; com muitas, tende a estabilizar sob a dinâmica da busca. Sob uma política fixa e amostras independentes, o erro típico da média decresce aproximadamente na ordem de $1/\sqrt{N(s)}$: reduzir o erro pela metade exige, aproximadamente, quadruplicar o número de simulações. No MCTS, essa relação deve ser lida como intuição, pois a política abaixo do nó muda ao longo da busca. Esses retornos decrescentes justificam distribuir simulações por vários nós, em vez de concentrá-las em um único.

Em jogos que admitem empate, como o xadrez, uma convenção comum é atualizar $W \ += 0,5$ em caso de empate. Assim, $Q(s)$ passa a refletir a utilidade esperada, e não apenas a taxa de vitória, sem alterar a estrutura do algoritmo.

2.1 Exemplo de árvore com contadores

A Figura 2 ilustra uma árvore MCTS parcial com quatro níveis. Cada nó exibe seus contadores no formato W/N : W é o número de simulações passando por aquele nó que resultaram em vitória de MAX, e N é o total de simulações que passaram por ele. Nós circulares representam estados MAX (é a vez de MAX jogar); nós retangulares representam estados MIN. A leitura correta de qualquer nó é sempre a mesma, independentemente do nível: “MAX venceu W das N simulações que passaram por aqui”. Assim, o nó raiz (20/60) indica que, das 60 simulações realizadas até agora, MAX venceu 20, resultando em $Q(\text{raiz}) = 20/60 \approx 0,33$. O nó MIN (14/35) indica que, das 35 simulações que desceram por aquele ramo, MAX venceu 14, e não MIN. Essa uniformidade é intencional e decorre da convenção estabelecida na Seção 2: $W(s)$ conta sempre vitórias de MAX, em qualquer nó. A consistência aritmética pode ser verificada em cada nível: os contadores do pai são sempre a soma dos contadores dos filhos. Por exemplo, $W(\text{raiz}) = 4 + 14 + 2 = 20$ e $N(\text{raiz}) = 15 + 35 + 10 = 60$; analogamente, $W(14/35) = 8 + 6 = 14$ e $N(14/35) = 20 + 15 = 35$. Observe também a distribuição assimétrica de visitas: o nó MIN (14/35) concentra 35 das 60 simulações totais, enquanto seus irmãos receberam apenas 15 e 10. Essa assimetria não é acidental; ela reflete o fato de que a política de seleção direcionou progressivamente mais simulações para o ramo que se mostrou mais promissor para MAX.

Interpretando alguns nós desta árvore (Figura 2):

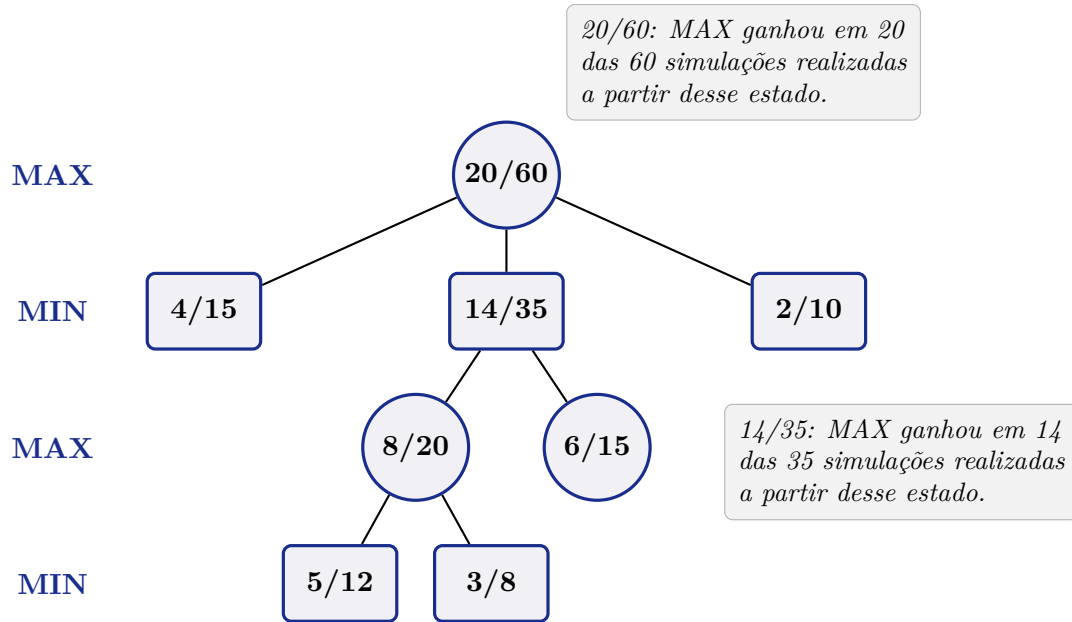


Figura 2: Árvore MCTS com quatro níveis. Em todos os nós, W/N representa vitórias de MAX sobre o total de simulações que passaram por aquele nó. A consistência aritmética é garantida em cada nível: $W(\text{pai}) = \sum_i W(\text{filho}_i)$ e $N(\text{pai}) = \sum_i N(\text{filho}_i)$.

- **Raiz (20/60):** a partir do estado raiz foram realizadas 60 simulações, das quais 20 resultaram em vitória para MAX. Portanto, $Q(\text{raiz}) = 20/60 \approx 0,33$.
- **Nó MIN (14/35):** trata-se de um estado em que é a vez de MIN jogar. Das 35 simulações passando por esse nó, 14 resultaram em vitória de MAX. Isso pode surpreender à primeira vista: por que um nó MIN teria alta taxa de vitória de MAX? A chave está em compreender *quem decidiu chegar até aqui*. Esse nó é filho da raiz (MAX), portanto foi MAX quem escolheu a ação que leva a esse estado. O MCTS selecionou esse caminho com frequência justamente porque ele se mostrou promissor para MAX. Em outras palavras: MAX prefere chegar nesse estado porque, a partir daqui, MAX tende a vencer, mesmo que seja MIN quem joga a seguir. Isso ilustra um ponto importante: $W(s)/N(s)$ alto em um nó não diz que o *jogador corrente* naquele nó está bem, mas sim que o *caminho que passou por aqui* costumou ser bom para MAX.
- **Nó MAX (6/15):** das 15 simulações passando por esse nó, apenas 6 resultaram em vitória de MAX. $Q = 6/15 = 0,40$. Apesar de ser um nó MAX, a taxa de vitória é moderada, indicando que esse é um estado razoavelmente difícil para MAX. Note que esse nó é filho de um nó MIN: foi MIN quem escolheu a ação que leva aqui; e MIN, naturalmente, tende a escolher estados desfavoráveis a MAX.
- **Nó MIN (3/8):** das 8 simulações passando por esse nó, apenas 3 resultaram em vitória de MAX. $Q = 3/8 = 0,375$. Esse nó tem sido pouco explorado em comparação com seu irmão (5/12); possivelmente porque a política de seleção o preteriu, uma vez que o caminho pelo nó irmão se mostrou ligeiramente mais promissor para MAX.

É importante não interpretar um $Q(s)$ alto como sinal de que o estado é bom para o jogador que decide em s . Como Q é sempre medido da perspectiva de MAX, um valor

alto significa somente que MAX obteve retorno alto nas simulações que passaram por esse estado. Portanto, em um nó MIN, esse mesmo valor é desfavorável a MIN sob a política considerada.

Pergunta 3

Dado um nó com contadores $W(s)$ e $N(s)$, qual é o valor de $Q(s)$? O que esse valor representa em termos de probabilidade de vitória? O que acontece com $Q(s)$ quando $N(s)$ aumenta muito?

Pergunta 4

Na árvore da Figura 2, o nó MIN com 14/35 tem $Q = 0,40$, enquanto o nó MIN com 4/15 tem $Q \approx 0,27$. Esses valores não significam que o primeiro nó é “bom para MIN”: $W(s)$ conta sempre vitórias de MAX. Explique então o que esses valores revelam sobre as *decisões de MAX* ao longo das simulações.

3 O Algoritmo MCTS: Quatro Passos

O MCTS repete um ciclo de quatro operações enquanto houver tempo ou até atingir um número fixo de iterações. Ao final, retorna a ação que levou ao filho da raiz com o maior $N(s)$.

O pseudocódigo abaixo apresenta a estrutura geral:

Algorithm 1 Monte Carlo Tree Search

```

1: função MCTS( $s_0$ ):
2:   Inicializa a árvore com um único nó raiz correspondente ao estado  $s_0$ , com  $N(s_0) = 0$ 
   e  $W(s_0) = 0$ .
3:   while houver tempo disponível do
4:      $\ell \leftarrow \text{SELECT}(s_0)$ 
5:     if  $\text{TERMINAL}(\ell)$  then
6:        $s' \leftarrow \ell$ 
7:        $\Delta \leftarrow \text{UTILITY}(\ell)$ 
8:     else
9:        $s' \leftarrow \text{EXPAND}(\ell)$ 
10:       $\Delta \leftarrow \text{SIMULATE}(s')$ 
11:    end if
12:     $\text{BACK-PROPAGATE}(s', \Delta)$ 
13:  end while
14: return  $\arg \max_a N(\text{RESULT}(s_0, a))$ ; em caso de empate, desempate por maior  $Q$  ou
   por uma regra fixa declarada.
```

Nota sobre a função Result: $\text{RESULT}(s, a)$ retorna o estado resultante de aplicar a ação a ao estado s ; essa função foi definida nas notas de Minimax e é reutilizada aqui sem modificação.

A função começa com uma árvore de nó único (linha 2). O laço principal executa iterações enquanto houver tempo. Para uma folha não terminal, cada iteração realiza

Select, Expand, Simulate e Back-Propagate; se a folha já for terminal, seu retorno é obtido diretamente e a iteração salta para Back-Propagate. Na última linha, a ação final é determinada: entre todos os filhos diretos da raiz, escolhe-se aquele com o maior $N(s)$, ou seja, aquele que recebeu mais simulações e, portanto, tem a estimativa mais confiável. Em caso de empate, a implementação deve declarar uma regra de desempate, por exemplo maior Q ou ordem fixa de ações.

Operacionalmente, cada iteração do MCTS responde a quatro perguntas: no passo **Select**, por onde descer na árvore já conhecida; em **Expand**, qual novo estado explorar; em **Simulate**, o que tende a acontecer ao jogar a partir desse estado; e em **Back-Propagate**, o que o resultado revela sobre as decisões anteriores. Cada pergunta é respondida uma única vez por iteração.

3.1 Passos do algoritmo

A seguir, cada um dos quatro passos é descrito em detalhe.

3.1.1 Passo 1: Select (Seleção)

Objetivo: percorrer a árvore da raiz até um nó que ainda possa ser expandido, escolhendo em cada nó interno qual filho visitar.

A partir da raiz, o algoritmo desce a árvore seguindo uma política de seleção (detalhada na Seção 4) que equilibra dois objetivos: visitar nós com alta estimativa de retorno para o jogador que decide (exploração) e visitar nós ainda pouco explorados (exploração). A formulação usual é:

continue descendo enquanto o nó atual for não terminal e estiver **completamente expandido**; ao chegar a um nó não terminal com alguma ação legal ainda não tentada, pare a seleção e passe à expansão.

Um nó terminal é aquele em que o jogo acabou. Um nó completamente expandido é um nó para o qual todas as ações legais já possuem filhos representados na árvore. Um nó ainda não completamente expandido não precisa escolher entre seus filhos já existentes: ele ainda tem ações novas a testar, e uma delas será adicionada no passo Expand. À medida que o MCTS realiza mais iterações, mais nós se tornam completamente expandidos e a seleção passa a descer mais fundo.

Na Figura 3, a política de seleção escolhe descer pelo nó MIN (14/35) e depois pelo nó MAX (6/15). Este último ainda não está completamente expandido: embora o jogo permita jogadas a partir dali, ao menos uma delas ainda não foi adicionada à árvore. É a partir desse nó que ocorrerá a expansão.

3.1.2 Passo 2: Expand (Expansão)

Objetivo: adicionar um novo nó filho ao nó folha selecionado.

A partir do nó folha ℓ , o algoritmo gera *um* filho ainda não representado na árvore, escolhido de acordo com algum critério (tipicamente, um filho aleatório ou o primeiro não visitado). Esse nó é adicionado à árvore com contadores $N = 0$ e $W = 0$.

Se o nó folha ℓ selecionado já for terminal, isto é, representar vitória, derrota ou empate, não há expansão possível porque o resultado já é conhecido. Nesse caso, Δ

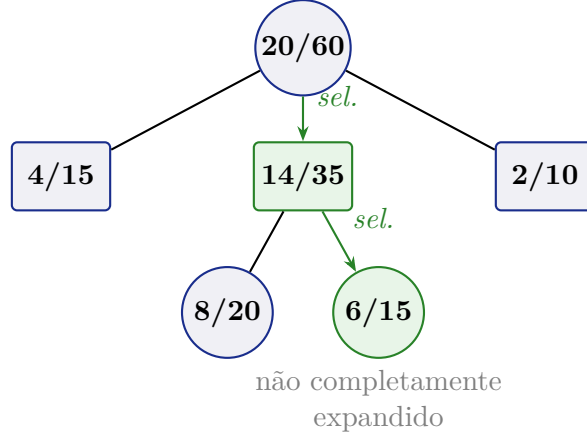


Figura 3: Passo Select. A política de seleção percorre a árvore descendo pelos nós em verde, guiada pela política UCT (Seção 4), enquanto os nós visitados estiverem completamente expandidos. O nó MAX (6/15) ainda possui pelo menos uma ação legal não representada na árvore; por isso a seleção para nele e a próxima etapa será expandi-lo.

é obtido diretamente do estado terminal e o algoritmo passa imediatamente ao Back-Propagate, sem executar Simulate. Esse caso é comum perto do final de uma partida e deve ser tratado explicitamente em qualquer implementação.

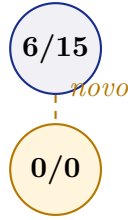


Figura 4: Passo Expand. Um novo filho (0/0, em laranja) é adicionado ao nó selecionado (6/15). Os contadores do novo nó são inicializados em zero, pois nenhuma simulação passou por ele ainda. A figura mostra apenas o novo filho; os demais filhos já existentes foram omitidos. Considerando todos os filhos armazenados, os contadores do pai permanecem iguais à soma dos contadores dos filhos.

Como mostra a Figura 4, o novo nó (0/0) representa um estado ainda desconhecido: nunca foi simulado. Exceto em estados terminais, a expansão faz a árvore crescer a cada iteração. No MCTS padrão, ela adiciona apenas *um* filho por iteração; isso preserva memória e direciona progressivamente o orçamento computacional às regiões mais promissoras. Variantes que expandem todos os filhos de uma vez existem, mas consomem mais memória e podem desperdiçar esforço em filhos de baixa qualidade.

3.1.3 Passo 3: Simulate (Simulação)

Objetivo: estimar o valor do nó recém-adicionado realizando um *playout* completo.

A partir do nó s' recém-criado (0/0), o algoritmo simula uma partida completa sem registrar os estados visitados na árvore. Ambos os jogadores escolhem ações de acordo com uma *política de simulação* (detalhada na Seção 5), que pode ser tão simples quanto escolher uniformemente entre as ações legais (*playout* aleatório) ou tão sofisticada quanto uma rede neural. A simulação continua até que um estado terminal seja atingido.

O resultado Δ da simulação é um valor numérico: $\Delta = 1$ se MAX venceu, $\Delta = 0$ se MIN venceu (ou em caso de empate, algum valor intermediário, como $\Delta = 0,5$, dependendo da convenção adotada).

Uma distinção fundamental é separar o que ocorre dentro da árvore do que ocorre fora dela. Nos passos Select e Expand, os nós são persistentes e acumulam os contadores W e N ao longo de muitas iterações. Já no passo Simulate, os estados visitados pelo *playout* são efêmeros: existem apenas para produzir o resultado terminal Δ e são descartados em seguida, sem receber contadores ou serem incorporados à árvore. Essa separação evita armazenar percursos que podem conter dezenas de estados e que não são necessários para a atualização; somente os nós persistentes no caminho raiz- s' são atualizados pelo Back-Propagate.

A Figura 5 ilustra a distinção central do passo Simulate: a linha tracejada separa dois mundos. Acima dela, os nós persistentes da árvore (raiz (20/60), nó MIN (14/35), nó MAX (6/15) e o nó recém-expandido (0/0)) têm contadores acumulados ao longo de muitas iterações. Abaixo dela, os nós do *playout* são efêmeros: representados em cinza tracejado para enfatizar que não são adicionados à árvore nem têm contadores criados para eles. A simulação desce por esses estados até atingir um nó terminal, de onde extrai o resultado $\Delta = 1$ (MAX venceu). Esse único valor é tudo que o playout precisa produzir; os estados intermediários são imediatamente descartados.

3.1.4 Passo 4: Back-Propagate (Retropropagação)

Objetivo: usar o resultado Δ da simulação para atualizar os contadores de todos os nós no caminho do nó s' até a raiz.

A partir de s' , o algoritmo sobe a árvore até a raiz, atualizando $N(s)$ e $W(s)$ em cada nó do caminho. Como $W(s)$ conta sempre vitórias de MAX (Seção 2), a regra de atualização é uniforme para todos os nós:

$$N(s) \leftarrow N(s) + 1 \quad (\text{sempre, em todos os nós do caminho})$$

$$W(s) \leftarrow W(s) + \Delta \quad (\text{em todos os nós do caminho})$$

No caso binário, $\Delta = 1$ se MAX venceu e $\Delta = 0$ se MAX perdeu. A simetria dessa regra é uma consequência direta da convenção uniforme: não é necessário distinguir nós MAX de nós MIN durante a retropropagação. Em cada nó do caminho, N é sempre incrementado e W é incrementado somente quando MAX venceu ($\Delta = 1$), independentemente do tipo do nó.

Vale notar por que faz sentido incrementar N e W também nos nós *acima* do nó recém-criado, e não apenas nele. Cada nó do caminho é co-responsável pelo resultado: a política de seleção escolheu descer por aquele caminho, e o resultado da simulação fornece informação sobre a qualidade dessa escolha. Atualizar todos os nós do caminho permite que iterações futuras tomem decisões mais informadas em todos os níveis da árvore, não apenas no nó recém-expandido.

A Figura 6 ilustra o passo Back-Propagate para o caso $\Delta = 1$ (MAX venceu a simulação). O caminho percorrido nesta iteração foi raiz $\rightarrow$ nó MIN (14/35) $\rightarrow$ nó MAX (6/15) $\rightarrow$ novo. Após a retropropagação, o novo nó passa de 0/0 para 1/1; o nó MAX do caminho passa de 6/15 para 7/16; o nó MIN do caminho passa de 14/35 para 15/36; e a raiz passa de 20/60 para 21/61. O nó MAX (8/20), filho do nó MIN mas fora do caminho selecionado nesta iteração, permanece inalterado — seus contadores só seriam atualizados se a política de seleção tivesse descido por ele.

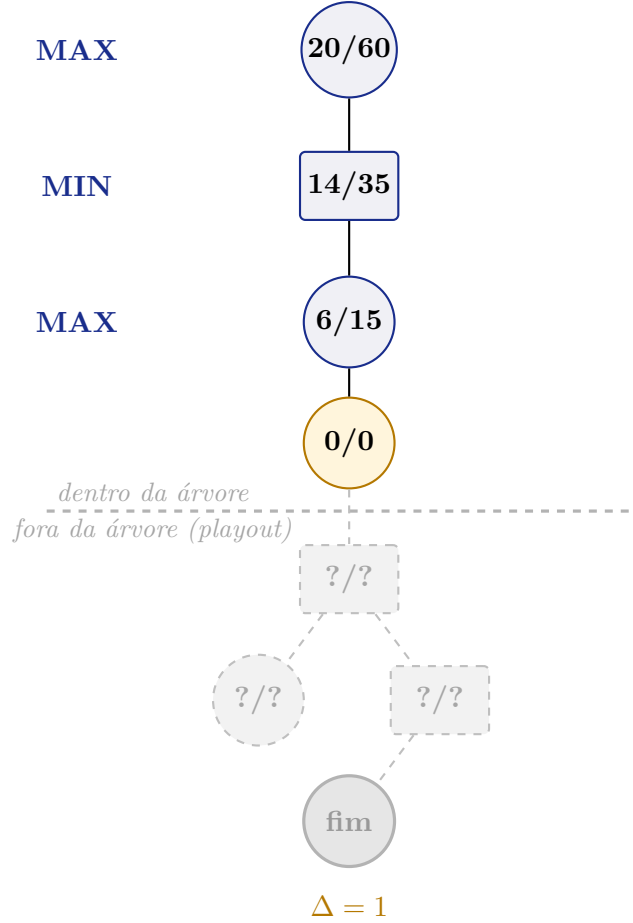


Figura 5: Passo Simulate. A partir do nó recém-criado (0/0, em laranja), o algoritmo realiza um *playout* completo até um estado terminal. Os nós acima da linha tracejada pertencem à árvore persistente; os nós abaixo (em cinza tracejado) são efêmeros e existem apenas durante a simulação, sendo descartados logo após. O resultado $\Delta = 1$ (vitória de MAX) será propagado de volta pela árvore no passo Back-Propagate.

3.2 Como o MCTS começa?

Até aqui, analisamos o MCTS a partir de árvores já parcialmente construídas. No entanto, é essencial entender o ponto de partida do algoritmo: *como a árvore nasce*.

Nas primeiras iterações, cria-se um filho da raiz por vez. Enquanto ainda houver ações legais não tentadas na raiz, a árvore cresce horizontalmente, explorando alternativas; depois, passa a crescer verticalmente, aprofundando os ramos selecionados. Esse comportamento decorre da regra de seleção e da prioridade atribuída a ações ainda não representadas.

Estado inicial

No início da execução, a árvore contém **apenas a raiz**, correspondente ao estado atual do jogo s_0 :

$$N(s_0) = 0, \quad W(s_0) = 0.$$

Não há filhos. Nenhuma ação foi explorada. Nenhuma simulação foi realizada. Nesse momento, o algoritmo não possui *qualquer informação* sobre a qualidade das ações dis-

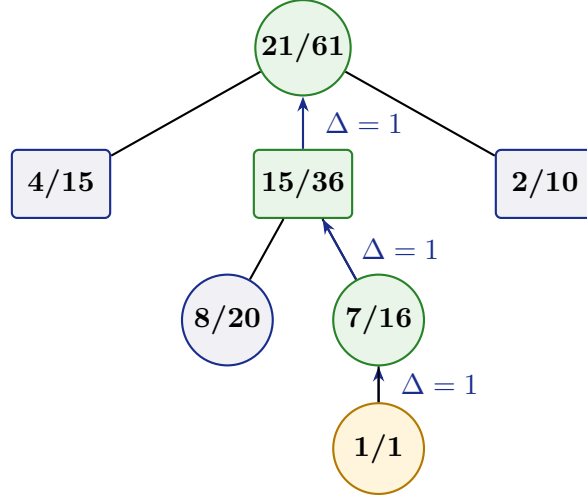


Figura 6: Passo Back-Propagate. Supondo $\Delta = 1$ (MAX venceu a simulação), os contadores de todos os nós no caminho raiz- s' são atualizados: $N \leftarrow N + 1$ e $W \leftarrow W + 1$. O nó recém-criado passa de 0/0 para 1/1; o nó MAX (6/15) passa a 7/16; o nó MIN (14/35) passa a 15/36; e a raiz (20/60) passa a 21/61. O nó MAX (8/20) *não* é atualizado pois não pertence ao caminho percorrido nesta iteração.

poníveis. No AlphaZero, os contadores da raiz também começam em zero; a rede fornece *priors* para as ações e uma estimativa de valor quando uma folha é avaliada.

Primeira iteração

A primeira iteração do MCTS constrói os primeiros dados a partir do zero.

Select: a raiz s_0 não possui filhos, portanto ela é imediatamente selecionada como folha:

$$\ell = s_0.$$

Expand: *exatamente um* filho legal de s_0 é adicionado à árvore; essa é uma decisão de projeto do algoritmo, não uma necessidade lógica. Denote esse novo estado por s_1 :

$$N(s_1) = 0, \quad W(s_1) = 0.$$

Agora a árvore contém dois nós: $s_0 \rightarrow s_1$.

Simulate: a partir de s_1 , executa-se um *playout* aleatório completo até um estado terminal. Para simplificar, assumimos jogos sem empate; o resultado é:

$$\Delta = \begin{cases} 1 & \text{se MAX vence,} \\ 0 & \text{se MIN vence.} \end{cases}$$

Em jogos com empate (como xadrez), é comum adotar $\Delta \in \{0, \frac{1}{2}, 1\}$.

Back-Propagate: os contadores são atualizados em cada nó do caminho de s_1 *de volta* até s_0 (sentido filho $\rightarrow$ raiz):

$$N(s) \leftarrow N(s) + 1, \quad W(s) \leftarrow W(s) + \Delta.$$

Após a atualização:

- s_1 : $W = \Delta$, $N = 1$;
- s_0 : $W = \Delta$, $N = 1$.

Segunda iteração: surgem as primeiras escolhas

Na segunda iteração, s_0 já possui um filho na árvore (s_1), mas seus demais filhos legais ainda não foram adicionados. Portanto, pela formulação usual, a seleção para novamente na raiz e o passo Expand adiciona mais um filho legal ainda não tentado.

Somente quando um nó está completamente expandido, isto é, quando todas as suas ações legais já possuem filhos na árvore, a política de seleção precisa escolher entre esses filhos. Nesse momento entram as fórmulas UCT/UCB1. No ciclo apresentado nesta nota, cada filho novo é simulado e retropropagado imediatamente; por isso, ele terá $N(s') = 1$ no início da iteração seguinte. Filhos já armazenados com $N(s') = 0$ podem ocorrer em variantes que criam vários filhos antes de simulá-los; nesse caso, como a UCB1 é indefinida, recebem prioridade máxima.

Interpretação

O MCTS não começa com decisões inteligentes. Ele começa *sem informação*. As primeiras iterações são dominadas por exploração:

- os valores $Q(s)$ são instáveis, baseados em poucas simulações;
- nós com ações legais ainda não tentadas são expandidos antes de a seleção precisar escolher entre todos os filhos;
- entre filhos já presentes na árvore, os não visitados ($N = 0$) têm prioridade máxima;
- a árvore cresce de forma *sequencial*: um filho de s_0 por iteração, até que todos sejam visitados.

À medida que o número de simulações aumenta:

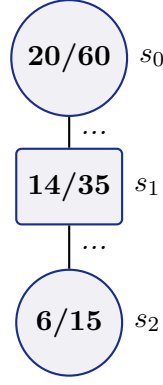
- as estimativas $Q(s)$ tornam-se mais confiáveis;
- a UCB1 passa a distinguir melhor os nós, direcionando esforço para regiões mais promissoras;
- a árvore cresce de forma *sequencial*: cada iteração adiciona exatamente um nó, não necessariamente filho da raiz.

Assim, o MCTS é um processo de *aprendizado incremental*: começa sem informação sobre a qualidade das ações e usa os resultados observados a cada iteração para refinar suas decisões. A árvore não é planejada de antemão; ela emerge das simulações.

3.3 Exemplo resolvido: uma iteração completa

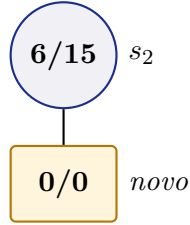
Considere a árvore abaixo antes de uma iteração. A iteração a ser executada resultará em vitória de MIN ($\Delta = 0$ para MAX).

Estado inicial:



Passo SELECT: a política de seleção escolhe descer por s_1 (14/35) e depois por s_2 (6/15). Como s_2 ainda possui uma ação legal sem filho representado, a descida para. Para manter a figura legível, os demais filhos já existentes de s_2 foram omitidos.

Passo EXPAND: s_2 recebe um novo filho MIN, rotulado *novo*, inicializado com $W = 0$, $N = 0$:



Passo SIMULATE: a partir de *novo* (0/0), a simulação é executada até um estado terminal. Os estados visitados durante a simulação *não* são adicionados à árvore. MIN vence: $\Delta = 0$.

Passo BACK-PROPAGATE: subindo de *novo* até s_0 , os contadores são atualizados. Como $\Delta = 0$ (MAX perdeu), W *não* é incrementado em nenhum nó; somente N é incrementado em todos:

Nó	Antes	Depois	Observação
novo (0/0)	$W = 0, N = 0$	$W = 0, N = 1$	$\Delta = 0$: só N incrementado
s_2 (6/15)	$W = 6, N = 15$	$W = 6, N = 16$	$\Delta = 0$: só N incrementado
s_1 (14/35)	$W = 14, N = 35$	$W = 14, N = 36$	$\Delta = 0$: só N incrementado
s_0 (20/60)	$W = 20, N = 60$	$W = 20, N = 61$	$\Delta = 0$: só N incrementado

Estado final: s_0 : 20/61; s_1 : 14/36; s_2 : 6/16; novo: 0/1.

Interpretação: como MAX perdeu ($\Delta = 0$), nenhum nó recebe incremento em W . Todos recebem incremento em N , pois a simulação passou por todos eles. A regra é uniforme: W é incrementado se e somente se MAX venceu, em qualquer nó do caminho.

Efeito sobre Q : observe o que acontece com Q nos nós do caminho após a atualização:

- s_2 : $Q = 6/16 = 0,375$ (era $6/15 = 0,400$). Q diminuiu.
- s_1 : $Q = 14/36 \approx 0,389$ (era $14/35 = 0,400$). Q também diminuiu.

- raiz: $Q = 20/61 \approx 0,328$ (era $20/60 \approx 0,333$). Q diminuiu.

Como esperado: uma derrota de MAX reduz as estimativas de todos os nós do caminho.

Pergunta 5

No passo Back-Propagate, qual é a regra de atualização de $W(s)$ e $N(s)$? A regra é diferente para nós MAX e nós MIN? Por quê?

Pergunta 6

Suponha que, em vez de MIN, foi MAX quem venceu a simulação ($\Delta = 1$). Refaça a tabela de Back-Propagate do exemplo acima com esse novo resultado. Quais nós têm W incrementado? Compare com o exemplo resolvido e explique a diferença.

3.4 Exemplo resolvido: árvore reduzida inspirada no Jogo da Velha

Esta seção acompanha quatro iterações de MCTS em uma árvore didática inspirada no jogo da velha. O jogador MAX é X e o jogador MIN é O. $W(s)$ conta sempre retornos de X, em qualquer nó da árvore.

A raiz s_0 corresponde ao tabuleiro vazio; é a vez de X jogar. Por brevidade, restringimos artificialmente o exemplo a dois filhos diretos da raiz: s_1 (X joga no centro) e s_2 (X joga no canto superior esquerdo). Em uma implementação completa do jogo da velha, as primeiras nove iterações tenderiam a expandir as nove ações legais da raiz antes de a UCB1 discriminar entre elas. Aqui usamos apenas duas ações para expor os cálculos em poucas linhas. Cada simulação consiste em completar a partida com jogadas aleatórias para ambos os jogadores até um estado terminal.

Iteração 1

Estado da árvore antes: apenas a raiz, $s_0 = 0/0$.

Select: a raiz não possui filhos na árvore; ela é imediatamente a folha selecionada.

Expand: um primeiro filho é adicionado: s_1 , correspondente a X jogando no centro. Contadores inicializados: $s_1 = 0/0$.



Simulate: a partir de s_1 , ambos os jogadores completam a partida com jogadas aleatórias. Resultado: X vence. $\Delta = 1$.

Back-Propagate: $\Delta = 1$ propaga de s_1 até s_0 :

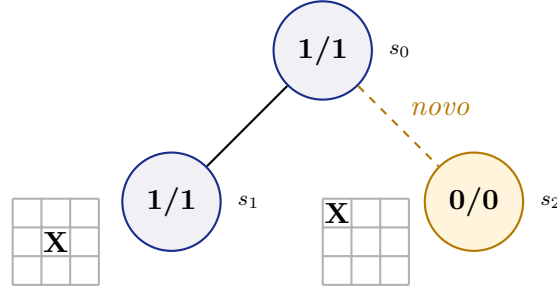
Nó	Antes	Depois
s_1	0/0	1/1
s_0 (raiz)	0/0	1/1

Iteração 2

Estado da árvore antes: $s_0 = 1/1$, filho $s_1 = 1/1$.

Select: a raiz tem um filho na árvore (s_1), mas seus demais filhos legais ainda não foram adicionados. Logo, a raiz ainda é uma folha da árvore atual.

Expand: um segundo filho é adicionado: s_2 , correspondente a X jogando no canto superior esquerdo. Contadores: $s_2 = 0/0$.



Simulate: playout aleatório a partir de s_2 . Resultado: O vence. $\Delta = 0$.

Back-Propagate: $\Delta = 0$; só N é incrementado.

Nó	Antes	Depois
s_2	0/0	0/1
s_0 (raiz)	1/1	1/2

Iteração 3

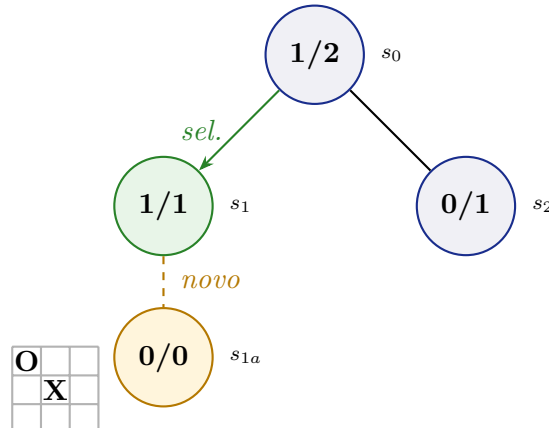
Estado da árvore antes: $s_0 = 1/2$; $s_1 = 1/1$; $s_2 = 0/1$.

Select: a raiz possui dois filhos com $N > 0$; a UCB1 é aplicada com $N(s_0) = 2$ e $C = \sqrt{2}$:

Filho	Q	Termo exploração	UCB1
s_1 (1/1)	1,000	$\sqrt{2} \cdot \sqrt{\ln 2/1} \approx 1,177$	2,177
s_2 (0/1)	0,000	$\sqrt{2} \cdot \sqrt{\ln 2/1} \approx 1,177$	1,177

s_1 é selecionado. Como s_1 não possui filhos na árvore, é a folha alcançada.

Expand: adiciona s_{1a} , filho de s_1 : O responde jogando no canto superior esquerdo. Contadores: $s_{1a} = 0/0$.



Simulate: playout a partir de s_{1a} . X vence. $\Delta = 1$.

Back-Propagate:

Nó	Antes	Depois
s_{1a}	0/0	1/1
s_1	1/1	2/2
s_0 (raiz)	1/2	2/3

Iteração 4

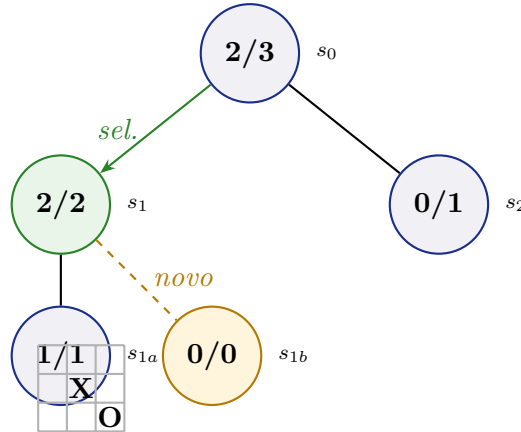
Estado da árvore antes: $s_0 = 2/3$; $s_1 = 2/2$; $s_2 = 0/1$.

Select: UCB1 com $N(s_0) = 3$ e $C = \sqrt{2}$:

Filho	Q	Termo exploração	UCB1
s_1 (2/2)	1,000	$\sqrt{2} \cdot \sqrt{\ln 3/2} \approx 1,048$	2,048
s_2 (0/1)	0,000	$\sqrt{2} \cdot \sqrt{\ln 3/1} \approx 1,482$	1,482

O bônus de exploração de s_2 é alto, mas não o suficiente para superar s_1 : $2,048 > 1,482$. Portanto, s_1 é selecionado. No exemplo reduzido, assumimos que s_1 ainda tem uma resposta de O não tentada; a seleção para em s_1 e a expansão adiciona esse novo filho.

Expand: adiciona s_{1b} , novo filho de s_1 : O responde jogando em um canto diferente. Contadores: $s_{1b} = 0/0$.



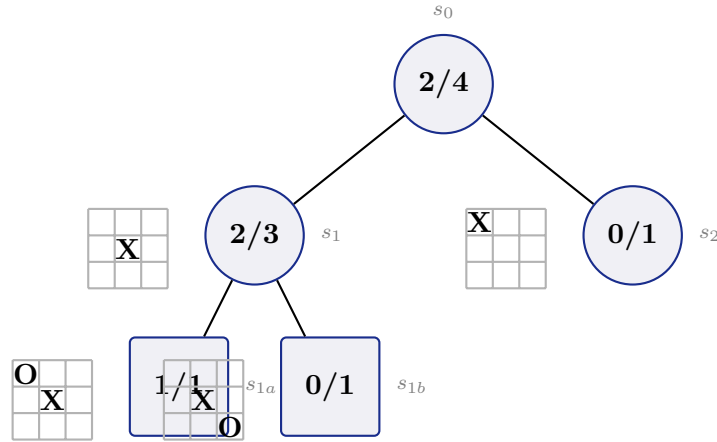
Simulate: playout a partir de s_{1b} . O vence. $\Delta = 0$.

Back-Propagate: $\Delta = 0$; só N é incrementado.

Nó	Antes	Depois
s_{1b}	0/0	0/1
s_1	2/2	2/3
s_0 (raiz)	2/3	2/4

Estado final e decisão

Após 4 iterações, a árvore está assim:



A decisão final é $\arg \max_a N(\text{RESULT}(s_0, a))$. O filho s_1 recebeu mais visitas que s_2 : $N(s_1) = 3$ contra $N(s_2) = 1$. O algoritmo escolhe s_1 : **X joga no centro**. Se houvesse empate em N , uma regra de desempate possível seria escolher o maior Q ; essa regra deve ser declarada explicitamente pela implementação.

Quatro iterações são insuficientes para uma decisão confiável em um jogo real. O jogo da velha tem nove ações legais na raiz, das quais apenas duas foram exploradas neste exemplo. Nas duas primeiras iterações, a árvore cresce horizontalmente ao adicionar filhos da raiz; na terceira, a UCB1 discrimina entre os filhos existentes e seleciona s_1 , que então recebe um neto; na quarta, o bônus de exploração de s_2 aumenta, mas ainda não supera a evidência acumulada em s_1 . Isso mostra que exploração não é alternância automática: ela compete numericamente com a exploração. Em uma implementação completa, os nove filhos da raiz seriam visitados nas iterações iniciais e, após centenas de iterações, a árvore cresceria vários níveis nas regiões mais promissoras, com estimativas mais confiáveis.

3.5 Critério de parada e resultado final

O laço principal termina quando o tempo disponível se esgota ou quando um número máximo de iterações é atingido. Nesse momento, a ação escolhida é:

$$a^* = \arg \max_{a \in \text{ACTIONS}(s_0)} N(\text{RESULT}(s_0, a))$$

Ou seja, entre os filhos diretos da raiz, escolhe-se aquele com o maior número de simulações. Se houver empate em N , o pseudocódigo precisa especificar uma regra adicional; uma escolha comum em exemplos didáticos é desempatar pelo maior Q , mas uma implementação também pode usar uma ordem fixa ou uma escolha aleatória controlada.

Em teoria, poderia-se retornar o filho com maior $Q(s) = W(s)/N(s)$. Na prática, usa-se frequentemente $\arg \max N$: um nó pode ter Q alto e N baixo, o que torna sua estimativa estatisticamente ruidosa, enquanto a política UCT tende a concentrar simulações nos nós considerados mais promissores. O maior número de visitas representa, portanto, a alternativa à qual a busca dedicou mais evidência ao término do orçamento, sem constituir garantia absoluta de maior valor em tempo finito.

Em jogos com mais de dois jogadores e objetivos próprios, o retorno de uma simulação deixa de ser um único valor Δ medido para MAX e passa a ser um vetor, por exemplo

$(\Delta_1, \Delta_2, \Delta_3)$. A expansão continua adicionando uma ação legal, mas a seleção deve avaliar cada filho segundo a utilidade do jogador que decide no nó, combinada a um termo de exploração. Ao fim do *playout*, a retropropagação atualiza, em cada nó do caminho, os componentes do retorno correspondentes aos três jogadores. A escolha da regra de seleção também precisa refletir a natureza do jogo: interesses puramente competitivos, cooperativos ou mistos não são descritos por uma única noção de “melhor para MAX”.

Pergunta 7

O MCTS foi projetado originalmente para jogos de dois jogadores de soma zero, mas pode ser adaptado para outros domínios, como jogos cooperativos, planejamento com incerteza ou otimização combinatória. Que modificações seriam necessárias no ciclo Select/Expand/Simulate/Back-Propagate para acomodar um jogo com três jogadores, em que cada um tem sua própria função de utilidade?

4 Política de Seleção e a Heurística UCB1

O passo Select percorre a árvore escolhendo, em cada nó interno, qual filho visitar. A escolha é governada por uma *política de seleção*. A qualidade dessa política é decisiva para a eficiência do MCTS: uma política ruim pode desperdiçar iterações em regiões sem interesse, enquanto uma boa política concentra os recursos computacionais onde são mais úteis.

4.1 O dilema exploração-exploração

A política de seleção enfrenta um dilema clássico em aprendizado por reforço e teoria da decisão:

- **Exploração** (*exploitation*): preferir nós com alta taxa de vitória $Q(s)$. Isso concentra as simulações nos ramos aparentemente bons, refinando as estimativas onde já há bons resultados.
- **Exploração** (*exploration*): preferir nós com poucas simulações ($N(s)$ baixo). Isso garante que nenhum ramo seja prematuramente descartado por falta de dados: um nó pouco explorado pode esconder a melhor jogada.

Uma política puramente explorativa ignora ramos pouco visitados e pode ficar presa em ótimos locais. A Figura 7 ilustra essa situação: o filho A acumulou muitas visitas e Q aparentemente alto, enquanto os filhos B e C foram descartados cedo após obterem Q baixo por azar nas primeiras simulações. Mesmo que C seja na verdade o melhor filho, uma política explorativa jamais o redescobrirá: seu Q baixo, consolidado com poucos dados, torna-o invisível para uma estratégia que só explora.

Uma política puramente exploratória visita todos os nós com frequência aproximadamente igual, desperdiçando iterações em ramos cuja inferioridade já foi estabelecida com alta confiança. A Figura 8 mostra esse caso: B e C têm Q muito baixo após dezenas de visitas, mas continuam recebendo simulações na mesma proporção que A.

A política ideal equilibra os dois objetivos, ajustando dinamicamente o foco conforme a evidência acumulada: aprofunda ramos promissores, mas reserva uma fração de esforço para revisitar regiões ainda incertas.

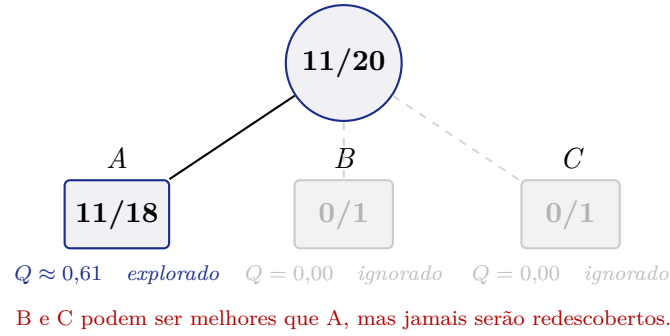


Figura 7: Política puramente explotativa. O filho A concentrou quase todas as visitas por apresentar Q inicialmente alto. Os filhos B e C foram visitados apenas uma vez cada e, por azar, não renderam nenhuma vitória de MAX nessas primeiras simulações. Com $Q = 0,00$, ambos são perpetuamente preteridos pela política explotativa, mesmo que, com mais simulações, revelassem ser superiores a A.

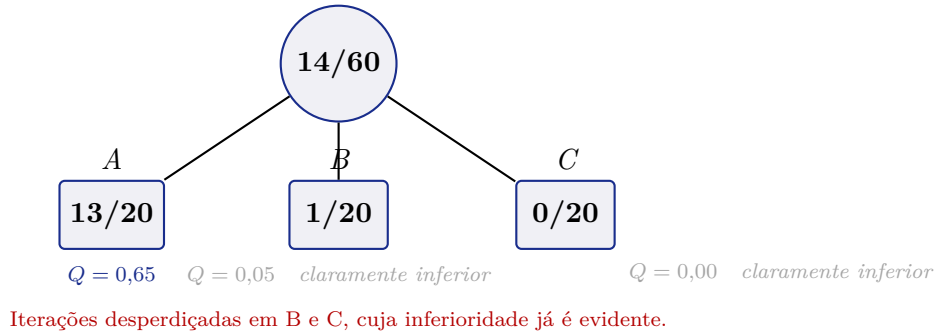


Figura 8: Política puramente exploratória. Após 60 simulações, os três filhos foram visitados com frequência uniforme (20 visitas cada). Os filhos B e C acumularam estimativas Q muito baixas — sua inferioridade já é estatisticamente evidente — mas continuam recebendo iterações na mesma proporção que A. O orçamento computacional é desperdiçado onde a decisão já está tomada.

Pergunta 8

Descreva o dilema exploração-explotação com suas próprias palavras. Por que uma política puramente explotativa pode falhar? Por que uma política puramente exploratória é ineficiente?

4.2 A heurística UCB1

A política de seleção mais comum no MCTS é a **UCT** (*Upper Confidence bounds applied to Trees*), que usa a heurística **UCB1** (*Upper Confidence Bound 1*). Para cada filho s' de um nó s , UCB1 computa:

$$\text{UCB1}(s') = \underbrace{\frac{W(s')}{N(s')}}_{\text{explotação}} + C \cdot \underbrace{\sqrt{\frac{\ln N(s)}{N(s')}}}_{\text{exploração}}$$

onde:

- $\frac{W(s')}{N(s')} = Q(s')$ é o **termo de exploração**: a taxa de vitória estimada de MAX a partir do nó s' .
- $C \cdot \sqrt{\frac{\ln N(s)}{N(s')}}$ é o **termo de exploração**. Para entender seu comportamento, vale analisar como ele reage a mudanças em $N(s')$:
 - Quando $N(s')$ é *pequeno* (nó pouco visitado), o denominador da fração sob a raiz é pequeno e o termo inteiro é *grande*; o UCB1 favorece visitar esse nó. Isso implementa a exploração: nós pouco conhecidos recebem um “bônus” que compensa a incerteza sobre seu valor real.
 - Quando $N(s')$ é *grande* (nó muito visitado), o denominador cresce e o termo de exploração *diminui*; o nó perde o bônus de novidade. A cada visita adicional, o bônus cai, de modo que outros nós menos visitados acabam sendo preferidos eventualmente.
 - O logaritmo $\ln N(s)$ no numerador garante que, mesmo para nós muito visitados, o bônus de exploração nunca caia abruptamente para zero: ele cresce devagar com o número total de visitas ao pai, assegurando que todos os filhos continuem recebendo atenção eventual.
- $C > 0$ é uma **constante de exploração** que controla o equilíbrio entre os dois termos. O valor $C = \sqrt{2}$ é derivado de argumentos teóricos no contexto do problema do *multi-armed bandit* (discutido abaixo); na prática, C costuma ser ajustado empiricamente para cada jogo.

A política UCT seleciona o filho conforme a perspectiva do jogador que decide no nó atual. Se Q é armazenado sempre como retorno de MAX, então, em um nó MAX, selecionamos:

$$s^* = \arg \max_{s' \in \text{filhos}(s)} \left[Q(s') + C \sqrt{\frac{\ln N(s)}{N(s')}} \right].$$

Em um nó MIN, o jogador que decide quer reduzir o retorno esperado de MAX. Uma forma consistente de escrever a regra, mantendo Q na perspectiva de MAX, é:

$$s^* = \arg \min_{s' \in \text{filhos}(s)} \left[Q(s') - C \sqrt{\frac{\ln N(s)}{N(s')}} \right].$$

O termo subtraído funciona como um bônus de exploração para MIN: filhos pouco visitados recebem um limite inferior mais otimista para MIN, pois ainda há incerteza sobre quão baixo pode ser o retorno de MAX.

Na prática, ao comparar filhos de um nó MAX, um Q maior favorece a exploração e um N menor recebe um bônus de exploração. Em um nó MIN, inverte-se o termo de valor: MIN prefere filhos com menor retorno esperado para MAX, mas ainda precisa explorar alternativas pouco testadas. A pergunta central é se um nó parece promissor por sua qualidade estimada ou apenas por ter sido pouco testado.

A UCB1 não é uma heurística ad hoc: ela foi derivada formalmente por Auer et al. [2002] para o problema de *multi-armed bandit*, no qual um agente deve escolher entre

k ações de recompensa desconhecida para maximizar o ganho acumulado. Ela possui garantias de *regret* sublinear: à medida que o número de iterações cresce, a perda acumulada por não escolher sempre a melhor ação aumenta apenas logaritmicamente. O valor $C = \sqrt{2}$ emerge dessa análise, e Kocsis and Szepesvári [2006] adaptaram UCB1 a árvores de jogo, originando o UCT. Para esta nota, importa sobretudo o significado dos dois termos e o papel de C .

Quando $N(s') = 0$, a expressão UCB1 é indefinida. Em variantes que podem manter filhos já armazenados sem simulação, um filho não visitado recebe prioridade máxima e é selecionado antes de qualquer filho visitado. Dessa forma, a política UCT garante que todos os filhos sejam visitados ao menos uma vez antes da aplicação direta da fórmula, coerentemente com a incerteza máxima associada a um nó sem observações. No ciclo padrão apresentado aqui, porém, o filho novo é simulado e retropropagado na própria iteração e, portanto, não permanece com $N = 0$.

Essa regra exige manter explícita a perspectiva do valor. Como $W(s')$ conta vitórias de MAX, $Q(s') = W(s')/N(s')$ mede o benefício daquele filho para MAX. Em um nó MAX, seleciona-se o filho com maior Q ; em um nó MIN, o filho com menor Q . Pode-se manter Q sempre na perspectiva de MAX e usar $\arg \max(Q + B)$ em nós MAX e $\arg \min(Q - B)$ em nós MIN, com $B = C\sqrt{\ln N(s)/N(s')}$; alternativamente, o valor pode ser convertido para a perspectiva do jogador que decide e então maximizado. Em nenhuma dessas convenções um nó MIN deve ser tratado como se maximizasse o retorno de MAX.

4.3 Exemplo resolvido: calculando UCB1

Considere a raiz com contadores 20/60 e seus três filhos (todos nós MIN):

Filho	$W(s')$	$N(s')$	$Q(s') = W/N$	Vez de quem joga
s_1	4	15	0,267	MIN
s_2	2	10	0,200	MIN
s_3	14	35	0,400	MIN

A raiz é um nó MAX; portanto, aplicamos $Q + B$ a cada filho e escolhemos o maior valor. O pai tem $N(s) = 60$. Usando $C = \sqrt{2} \approx 1,414$ e $\ln 60 \approx 4,094$:

Para s_1 (4/15):

$$\begin{aligned}
 \text{UCB1}(s_1) &= \frac{4}{15} + \sqrt{2} \cdot \sqrt{\frac{\ln 60}{15}} \\
 &= 0,267 + 1,414 \cdot \sqrt{\frac{4,094}{15}} \\
 &= 0,267 + 1,414 \cdot 0,522 \\
 &= 0,267 + 0,739 \\
 &= \mathbf{1,006}
 \end{aligned}$$

Para s_2 (2/10):

$$\begin{aligned}
\text{UCB1}(s_2) &= \frac{2}{10} + \sqrt{2} \cdot \sqrt{\frac{\ln 60}{10}} \\
&= 0,200 + 1,414 \cdot \sqrt{\frac{4,094}{10}} \\
&= 0,200 + 1,414 \cdot 0,640 \\
&= 0,200 + 0,905 \\
&= \mathbf{1,105}
\end{aligned}$$

Para s_3 (14/35):

$$\begin{aligned}
\text{UCB1}(s_3) &= \frac{14}{35} + \sqrt{2} \cdot \sqrt{\frac{\ln 60}{35}} \\
&= 0,400 + 1,414 \cdot \sqrt{\frac{4,094}{35}} \\
&= 0,400 + 1,414 \cdot 0,342 \\
&= 0,400 + 0,484 \\
&= \mathbf{0,884}
\end{aligned}$$

Tabela resumo:

Filho	$Q(s')$	Termo de exploração	UCB1	
s_1 (4/15)	0,267	0,739	1,006	
s_2 (2/10)	0,200	0,905	1,105	← selecionado
s_3 (14/35)	0,400	0,484	0,884	

Interpretação: s_2 é selecionado, apesar de ter o menor Q entre os três filhos. O motivo é que s_2 é o nó menos visitado ($N = 10$), o que lhe confere o maior bônus de exploração (0,905). Isso ilustra uma propriedade importante do UCB1: um nó com estimativa de qualidade modesta mas alta incerteza pode superar um nó aparentemente melhor que já foi amplamente testado. Note que s_3 , o filho com maior Q (0,400), obtém o menor UCB1 exatamente por ser o mais visitado ($N = 35$): seu bônus de exploração é o menor dos três. À medida que s_2 for visitado mais vezes, seu bônus de exploração diminuirá e o foco se deslocará naturalmente para outros filhos. Esse é exatamente o comportamento desejado: o UCB1 redistribui o foco quando um nó já foi suficientemente explorado.

O termo de exploração diminui para filhos muito visitados, mas não elimina a exploração em um orçamento finito. No problema clássico de bandits estacionários, UCB1 continua visitando braços subótimos tipicamente em ordem logarítmica, $O(\log n)$, enquanto o melhor braço concentra a maior parte das visitas. Em árvores de jogo, a mesma intuição produz uma busca que concentra simulações nos ramos promissores sem abandonar por completo ramos ainda incertos. O parâmetro C controla esse equilíbrio: valores altos privilegiam exploração, o que pode ser útil com poucas simulações ou muitas ações não testadas; valores baixos privilegiam exploração quando já há mais evidência. Na prática, C é ajustado empiricamente para o domínio, e $C = \sqrt{2}$ é um ponto de partida comum.

Pergunta 9

Na fórmula UCB1, o que acontece com o termo de exploração quando $N(s')$ é muito grande? E quando $N(s')$ é muito pequeno? Como esse comportamento implementa o equilíbrio exploração-exploração?

Pergunta 10

Suponha que o nó s_2 (2/10) receba mais 10 simulações sem nenhuma vitória adicional, passando para 2/20. Recalcule UCB1 para os três filhos (considere que o pai passa a ter $N = 70$). O nó selecionado muda? O que isso demonstra sobre o comportamento do UCB1 ao longo do tempo?

Pergunta 11

O MCTS comporta-se de forma muito diferente nas primeiras iterações (quando a árvore tem poucos nós e a exploração domina) e nas últimas (quando a árvore é densa e a exploração domina). Descreva como a distribuição de visitas entre os filhos da raiz evolui ao longo das iterações. O que você esperaria observar se plotasse $N(s')$ para cada filho da raiz em função do número total de iterações?

5 Política de Simulação

O passo Simulate realiza um *playout*: a partir do nó recém-expandido s' , ambos os jogadores fazem escolhas de acordo com uma *política de simulação* até o fim da partida. O resultado terminal é o único sinal de avaliação utilizado.

A política de simulação é independente da política de seleção: a seleção opera *dentro* da árvore (sobre nós persistentes com contadores acumulados), enquanto a simulação opera *fora* da árvore (sem registrar estados, descartando o percurso ao final). Essa separação de responsabilidades permite que cada política seja projetada e ajustada independentemente.

5.1 Política de simulação aleatória

A política mais simples é a *política uniforme aleatória*: em cada estado, escolhe-se uma ação legal aleatoriamente com probabilidade uniforme. Essa política é computacionalmente barata e suficiente quando o jogo é relativamente simples ou quando o número de iterações é grande o suficiente para compensar o ruído das simulações.

Há um compromisso entre a qualidade da política de simulação e o custo de cada iteração. Uma política aleatória é barata e permite muitas iterações por segundo, mas produz *playouts* ruidosos porque os dois jogadores podem realizar escolhas sem sentido estratégico. Uma política heurística mais sofisticada produz sinais de melhor qualidade, mas reduz o número de iterações que cabe no mesmo orçamento de tempo. Em jogos simples, muitas simulações podem compensar o ruído; em jogos complexos como Go, uma política de melhor qualidade pode ser necessária mesmo que permita menos simulações.

5.2 Políticas de simulação baseadas em heurísticas

Em jogos mais complexos, uma política aleatória gera simulações de baixa qualidade: as ações escolhidas podem ser francamente ruins, e o resultado da simulação fornece um sinal ruidoso sobre o valor do nó. Por que isso acontece? Quando ambos os jogadores fazem escolhas aleatórias, a partida simulada não se assemelha a uma partida real entre jogadores competentes. Um estado que seria claramente perdedor contra um oponente forte pode parecer vencedor em simulações aleatórias, simplesmente porque o oponente “aleatório” não percebe as ameaças.

Uma alternativa é usar heurísticas específicas do domínio para guiar as escolhas durante a simulação:

- **Xadrez:** priorizar jogadas que capturam peças desprotegidas do oponente.
- **Damas:** priorizar jogadas que ocupam as casas dos cantos do tabuleiro.
- **Go:** priorizar jogadas que conectam grupos de pedras amigáveis ou separam grupos adversários.

Esse tipo de política codifica conhecimento especializado diretamente na simulação, melhorando a qualidade do sinal de avaliação sem aumentar o custo computacional de forma proibitiva.

Pergunta 12

Uma política de simulação puramente aleatória funciona melhor em jogos simples (como o jogo da velha) do que em jogos complexos (como o Go). Por quê? O que torna o sinal de uma simulação aleatória mais ruidoso em jogos complexos?

5.3 Políticas aprendidas por self-play

Uma alternativa ao design manual de heurísticas é *aprender* componentes da busca automaticamente. A técnica de *self-play* consiste em fazer o agente jogar contra si mesmo, usando os resultados das partidas para ajustar sua política e sua estimativa de valor. O ciclo é iterativo e se fecha sobre si mesmo:

1. **Geração de dados:** o agente joga muitas partidas contra si mesmo usando a política atual (tipicamente implementada como uma rede neural). Em cada posição, o MCTS é executado por um número fixo de iterações e produz uma *distribuição de visitas* $\pi(a | s) \propto N(\text{RESULT}(s, a))$ sobre as ações, essencialmente, a fração de simulações destinada a cada ação. O resultado final da partida (+1, -1 ou 0) é registrado.
2. **Treinamento da rede:** os dados gerados servem como alvo para treinar (ou refinar) a rede neural. A *rede de política* é treinada para prever a distribuição π produzida pelo MCTS; a *rede de valor* é treinada para prever o resultado final da partida a partir de cada estado visitado. Em ambos os casos, a rede está sendo supervisionada por sinais gerados pelo próprio MCTS, não por especialistas humanos.
3. **Melhoria do agente:** a rede atualizada passa a guiar a busca nas próximas partidas, fechando o ciclo. Em sistemas modernos, ela fornece tanto probabilidades-prior para ações quanto uma estimativa de valor para estados.

A força dessa abordagem está na retroalimentação: à medida que a rede melhora, a busca guiada por ela produz dados de maior qualidade, que por sua vez melhoram ainda mais a rede. Esse ciclo permite que o agente descubra estratégias progressivamente mais sofisticadas sem nenhum conhecimento humano além das regras do jogo.

O **AlphaGo** [Silver et al., 2016] combinou MCTS com redes neurais treinadas em partidas humanas e por self-play e derrotou o campeão mundial de Go Lee Sedol por 4 a 1. O **AlphaZero** [Silver et al., 2017] generalizou essa abordagem: sem usar uma base de partidas humanas, aprendeu xadrez, shogi e Go por self-play e superou os melhores programas especializados em cada jogo.

O AlphaZero preserva o ciclo descrito acima, mas difere do MCTS clássico com *playouts* aleatórios. Sua rede neural recebe um estado e produz uma distribuição-prior sobre ações legais e uma estimativa de valor. A seleção na árvore usa PUCT, uma variante que combina os valores acumulados da busca com esses priors. Ao alcançar uma folha, em geral o algoritmo não executa um *playout* aleatório até um terminal: avalia a folha com a rede e retropropaga esse valor. Assim, a rede não apenas melhora a política de simulação, mas modifica o próprio papel da simulação e torna a busca mais eficiente.

Pergunta 13

O AlphaZero combina MCTS com redes neurais treinadas por self-play. A rede fornece probabilidades-prior para as ações e uma estimativa de valor para os estados; a busca usa essas informações para selecionar e avaliar folhas. Discuta: se a rede neural for suficientemente boa, seria possível eliminar a busca completamente e usar apenas as estimativas da rede? Quais seriam os riscos?

6 Resumo e Comparação com o Minimax

A tabela abaixo sintetiza as principais diferenças entre o Minimax (com poda alfa-beta) e o MCTS:

Aspecto	Minimax / Alfa-Beta	MCTS
Avaliação de estados	Função heurística $EVAL(s)$ ou busca completa até terminais	MCTS básico usa resultados de simulações; variantes modernas podem usar heurísticas ou redes de valor
Crescimento da árvore	Uniforme até profundidade d	Assimétrico, guiado por UCB1
Adequação ao fator b	Exponencial em d : inviável para b grande	Concentra simulações nos ramos promissores; adapta-se melhor a b grande
Garantia de otimalidade	Sim apenas com busca completa ou avaliação exata; com profundidade limitada depende da qualidade de $EVAL$	Sem garantia de decisão ótima em orçamento finito; UCT possui resultados assintóticos sob hipóteses adequadas
Comportamento anytime	Minimax de profundidade fixa não é naturalmente anytime; aprofundamento iterativo pode torná-lo interrompível	Sim (melhora com mais iterações)
Conhecimento de domínio	Necessário quando se usa $EVAL(s)$ manual	MCTS básico pode usar política aleatória em jogos simples; desempenho em jogos complexos depende fortemente da política de simulação ou de modelos aprendidos
Complexidade de espaço	$O(bd)$	$O(N_{\text{iter}})$ (cresce com as iterações)

É importante ressaltar que MCTS e Minimax não são mutuamente exclusivos. Sistemas modernos de alto desempenho, como o AlphaZero, combinam MCTS (para a busca) com redes neurais profundas (para fornecer priors de política e estimativas de valor), obtendo o melhor dos dois mundos: a eficiência do MCTS em explorar seletivamente o espaço de estados, e a qualidade das estimativas fornecidas por modelos de aprendizado profundo.

Pergunta 14

Em orçamento finito, o MCTS não garante retornar a jogada ótima. Em contrapartida, Minimax só garante otimalidade quando a busca é completa ou quando a avaliação usada nos cortes é exata. Discuta essa troca: em quais situações a ausência de garantia em orçamento finito seria inaceitável? Em quais seria irrelevante na prática? Como resultados assintóticos de convergência do UCT mudam, ou não mudam, essa análise?

Pergunta 15

Cite duas situações em que o MCTS tem vantagem sobre o Minimax puro, e uma situação em que o Minimax pode ser preferível.

Pergunta 16

O MCTS é descrito como um algoritmo *anytime*. O que isso significa? Por que o Minimax com profundidade fixa não é anytime? Qual característica do ciclo Select/Expand/Simulate/Back-Propagate garante esse comportamento?

7 Atividades Intra-Classe

As atividades abaixo devem ser resolvidas em grupo, com foco em explicar o raciocínio passo a passo, e não apenas em obter a resposta final.

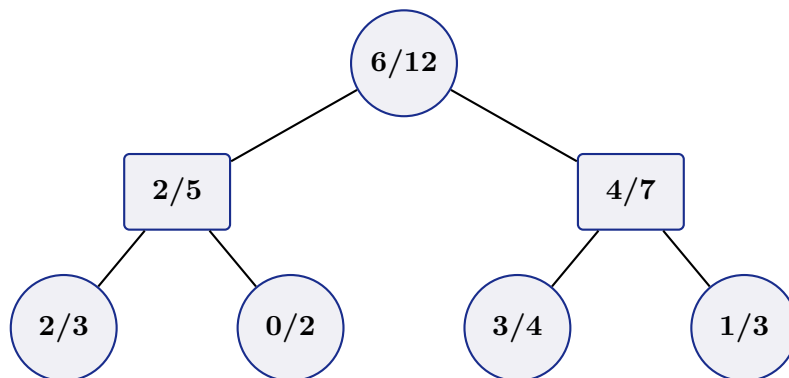
7.1 Aquecimento

Antes das atividades principais, responda rapidamente:

- (a) Um nó tem contadores $W = 8$ e $N = 20$. Qual é $Q(s)$? O que esse valor representa?
- (b) Por que o MCTS não precisa de uma função $EVAL(s)$ explícita como o H-Minimax?
- (c) O que acontece com o termo de exploração da UCB1 quando $N(s')$ cresce indefinidamente?
- (d) Um nó MIN tem $W = 15$ e $N = 20$, resultando em $Q = 0,75$. Isso significa que MIN está “ganhando” a partir desse estado? Explique à luz da convenção de $W(s)$.

7.2 Exercício: simulação manual do MCTS

Considere a seguinte árvore MCTS parcial (valores W/N). A raiz é um nó MAX.



- (a) Para cada filho da raiz (m1 e m2), calcule $Q + B$ usando

$$C = \sqrt{2}, \quad B = C \sqrt{\frac{\ln N(\text{raiz})}{N(\text{filho})}}, \quad N(\text{raiz}) = 12.$$

Como a raiz é MAX, qual filho é selecionado?

- (b) Continue descendo a árvore. O nó selecionado no item anterior é MIN; portanto, compare seus filhos usando $Q - B$ e escolha o menor valor. Qual nó folha é alcançado? *Dica: observe o efeito do bônus de exploração no nó a2 (0/2), que tem $Q = 0$ mas $N > 0$.*
- (c) Suponha que o EXPAND gere um novo filho com $W = 0$, $N = 0$ a partir do nó folha. A SIMULATE produz uma vitória de MAX ($\Delta = 1$). Execute o BACK-PROPAGATE e mostre os contadores atualizados em cada nó do caminho raiz-folha-novo.
- (d) Após essa iteração, qual ação a política UCT escolheria se fosse encerrar o algoritmo agora (i.e., qual filho da raiz tem maior N)?
- (e) *Cenário extra:* suponha que, em vez de executar o Simulate, o nó folha selecionado no item (b) fosse um estado terminal de vitória de MAX. Como o algoritmo se comportaria a partir daí? Qual seria a diferença em relação ao fluxo normal?

7.3 Exercício: UCB1 e o dilema exploração-exploração

Considere um nó MAX com $N = 50$ e três filhos:

Filho	W	N
A	18	20
B	5	25
C	0	5

- (a) Calcule UCB1 para cada filho com $C = \sqrt{2}$. Qual é selecionado?
- (b) O filho C tem $Q = 0$ (nenhuma vitória). Por que o UCB1 ainda pode selecioná-lo? Em que condições isso ocorre?
- (c) Suponha que o parâmetro C seja aumentado para $C = 3$. Recalcule. O nó selecionado muda? O que isso diz sobre o papel de C ?
- (d) Qual seria o nó selecionado se $C = 0$ (sem exploração)? Que problema isso causaria a longo prazo?

7.4 Projeto de política de simulação

Para o jogo da velha (3×3):

- (a) Proponha uma política de simulação não-aleatória. Defina explicitamente a regra de escolha de ações para ambos os jogadores e justifique por que ela deve produzir simulações de melhor qualidade do que uma política uniforme.
- (b) Sua política é específica ao domínio (jogo da velha) ou poderia ser aplicada a outros jogos? Discuta.
- (c) Descreva um cenário em que uma política de simulação de má qualidade leva o MCTS a tomar uma decisão errada sob um orçamento finito, mesmo que grande.
- (d) Discuta o trade-off entre qualidade da política e custo por iteração: em que circunstâncias vale a pena usar uma política mais cara?

Referências

- Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002.
- Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *European Conference on Machine Learning (ECML)*, pages 282–293. Springer, 2006.
- David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.